

Claude Code Prompt — Actuate Build Increment 1

Paste everything below the line into Claude Code (in an empty repo, or point it at your existing Actuate repo). It references three design docs — put `Actuate_Implementation_Spec_v3.md`, `Actuate_AWS_Architecture.md`, and `Actuate_Architecture_v3.md` in the repo root (or paste their contents) so the agent can read them.

You are helping build **Actuate**, a production, CLI/API-first multimodal data pipeline that turns raw robot-capture data (egocentric head-mounted, UMI handheld gripper, stereo, instrumented-glove, teleoperated-robot, DexUMI exoskeleton) into training-ready datasets (LeRobot v3 / RLDS) for frontier robotics labs. Read `Actuate_Implementation_Spec_v3.md` and `Actuate_AWS_Architecture.md` in full before writing any code — they are the source of truth for the package structure, the canonical schema, the AWS storage/DB design, and the build discipline. Do not invent architecture that contradicts them.

Scope of THIS increment (Increment 1 only — do not build beyond it)

Build the foundation that everything else compiles against, and stop for review. Exactly these five things:

1. **Repo skeleton, CLI/API-first.** The package layout from Implementation Spec §2.1: an importable `actuate/` library as the source of truth, with `cli/` (Typer) and `service/` (FastAPI) as thin stubs that import from the library and never the reverse. Set up `pyproject.toml` (use `uv`), optional extras (`actuate[perception]`, `actuate[retarget]`, `actuate[sim]`, `actuate[aws]`), and an **import-linter contract in CI** that enforces the dependency direction (nothing in a layer package may import from `cli/` or `service/`; layers import only from `schema/`, `io/`, `config/`, `catalog/`, and each other in pipeline order).
2. **The frozen canonical schema** (`actuate/schema/`), exactly per Implementation Spec §3 — the per-frame and per-episode/segment records, using `msgspec.Struct` or Pydantic v2, with `schema_version=1`. Include the MANO-as-intermediate + retargeted-joints-as-delivered-target decision, both control modes, both finger-action representations, the $\pi 0.7$ episode-metadata slots, `strategy_alignment`, `consent/pii_status`, `tier`, and `effective_hours`.

This is the freeze point; treat it as an API contract.

3. **Storage + catalog abstraction** (`actuate/io/` and `actuate/catalog/`) so every layer reads/writes through one interface — never raw `boto3` scattered around. `io/` wraps S3 blob I/O (Zarr + Parquet + chunked-MP4 refs via `fsspec/s3fs`, with a local-filesystem backend for dev/tests). `catalog/` wraps the Postgres catalog (SQLAlchemy models mirroring the schema's episode/metadata fields per AWS Architecture §3, Alembic migrations) — the query surface for episodes/tasks/embodiments/scenes/consent/quality/jobs. **Blobs in S3, everything queryable in Postgres with an S3 URI pointer.** Include a `consent`-aware write guard: the delivery-write path refuses any episode whose `consent/pii_status` is not `passed` (fail-closed).
4. **AWS IaC for the storage + DB backbone** (`infra/`, **AWS CDK in Python**), per AWS Architecture §2, §3, §6 — but ONLY the `StorageStack` and `DataStack` this increment: the four S3 buckets with the **consent boundary** (Block Public Access, SSE-KMS, TLS-only, lifecycle rules, and IAM/bucket policies such that only the packaging role can write to `actuate-delivery-*` and raw/work roles have no delivery `PutObject`), the KMS CMK, RDS/Aurora-Serverless-v2 Postgres, and Secrets Manager for DB creds. Do NOT build Batch/Step Functions/Fargate yet — those are a later increment. Make the stacks `cdk synth`-clean; parameterize `dev/prod` via context.
5. **Tests + CI**, per the project's verification discipline (below).

Stub only (do not implement logic this increment): the CLI command groups from Implementation Spec §2.2 (they should parse and print "not implemented"), the FastAPI app (health check + stubbed routes), and empty layer packages `ingest/` `perception/` `fusion/` `canonical/` `certify/` `retarget/` `language/` `package/` `feedback/` with docstrings stating their spec section.

Working discipline — follow this exactly (it is how this project has been built successfully)

- **CLI/API-first is a hard rule.** If a capability only works through the CLI or the service, it is wrong. Library first; CLI and service are thin consumers. The import-linter contract must actually run in CI and fail if violated — verify it fails on a deliberately bad import, then remove that.
- **Verify against real data / real behavior, never trust mocks or "looks right."** For this foundation increment "real" means: the schema round-trips a realistic episode object → Parquet/Zarr (local backend) → reload with bit-exact field recovery; the catalog performs a real query against a **real Postgres** (spin up Postgres in a test

container / `testcontainers`, not a mock); and `cdk synth` actually produces valid templates.

- **Every test that is meant to prove correctness must be confirmed to FAIL against a broken version, not just pass against the current one.** Concretely, include and demonstrate: (a) a schema-migration test that FAILS when a required field is dropped; (b) a consent-guard test that FAILS (i.e., data leaks) if the fail-closed check is removed — prove it catches the "auto-consent workaround" bug class; (c) the import-linter contract failing on a bad import. In your final report, show the before/after of each: broken → test red, fixed → test green.
- **Be honest about status.** For each of the five deliverables, report precisely: tested-against-real-data vs. only-unit-tested vs. only-written. Do not claim something works because it compiles or "looks right." If you didn't run it, say so.
- **Do not scope-creep.** No perception models, no retargeting, no exporters, no Batch/Fargate this increment. If you find yourself needing them, stop and note it for the next increment.
- **No hardcoded credentials or secrets** anywhere; use Secrets Manager / env for config. Do not commit real AWS account IDs.

Deliverables to produce

- Working `actuate/` package installable via `uv`, importable, with the frozen schema, `io/`, `catalog/`, config (Pydantic-settings + layered YAML + rig/embodiment registries), and stubbed `cli/`, `service/`, and empty layer packages.
- `infra/` CDK-Python app with `StorageStack` + `DataStack`, `cdk synth`-clean.
- `tests/` (unit + integration with real Postgres via `testcontainers`) and CI config (lint, import-linter contract, tests, `cdk synth`).
- A `README.md` with: how to install, how to run tests, how to `cdk synth/deploy` the two stacks, and the exact command to bring up the consent boundary first.
- A short **`STATUS.md`** with the honest per-deliverable status (tested-real / unit-only / written-only) and the three broken-vs-fixed test demonstrations.

Stop condition

After these five things, **stop and summarize for review** — do not continue to Increment 2 (which will be: canonical `build` from existing v1 outputs, then the LeRobot v3 exporter with the load+train verification gate). List anything you had to assume or defer.

Begin by reading the two design docs, then propose the concrete file tree and the schema module you'll write, and wait for my go-ahead before generating the full implementation.